

1 Les fondations du programmation Python

Python est réputé pour sa syntaxe claire et lisible, ce qui en fait un excellent langage de programmation pour les débutants. Avant de plonger dans des projets plus complexes, il est essentiel de maîtriser les instructions de base qui constituent les fondements de la programmation en Python.

2 Variables et Types de Données

En Python, vous pouvez facilement déclarer des variables sans spécifier explicitement leur type de données. Le type est automatiquement attribué en fonction de la valeur assignée :

Exemple :

```
1
2 message = "Bonjour"
3 nombre = 25
4
```

Ce que vous devez retenir :

- ① Pour déclarer une variable, il faut la donner un nom : "message" et "nombre" comme l'exemple précédent le montre.
- ② Si vous déclarer une variable, il faut systématiquement l'attribuer une valeur, pour que Python comprenne le type du variable.

Pour voir le type d'une valeur en Python, on utilise la fonction **type** :

```
1 message = "Bonjour"
2 print(type(message))
```

À cette stade peut-être la notion de **fonction** ne vous parle pas, mais ne vous en fait pas une simple copie coller fera largement l'affaire.

Types de Données :

```
1 nom = "Ali" # str (String, Chaîne de caractère)
2 age=19 #int (Entier)
3 taille= 1.70 # flottant(reel)
4 etudiant=True # Boolleen
```

Comme vous le remarquez, il y a du code précédé par le symbole dièse. C'est ce qu'on appelle un commentaire, cette notion existe sur tous les langages de programmation mais il est exprimé d'une manière différente. Par exemple en **Python** on utilise le symbole dièse et double slash et en **Java**, bref vous avez compris l'idée. Et là vous vous demandez peut-être **ce quoi cet histoire commentaire**,

eh bien on utilise les commentaires pour commenter notre code et ce dernier ne sera pas pris en considération lors de l'exécution.

Exemple :

```
1 # Je declare une variable de type <str> et je l'affecte a la valeur <Ali>
2 nom = "Ali"
```

3 Opérations de base et Structures de contrôle

3.1 Opérations de base

Dans la section précédente, on a vu comment déclarer des variables, connaître leurs types et même faire des commentaires. À présent, nous allons apprendre les bases des opérations arithmétiques et de comparaison en Python. Pas si vite, c'est quoi une opération arithmétique? Eh bien, ce sont des opérations mathématiques fondamentales que vous pouvez réaliser facilement avec des nombres.

❶ Opération arithmétique

```
1 # Operation arithmetique
2 x=1
3 y=3.6
4 print(x + y) # L'addition de x et y
5 print(x - y) # La soustraction de x et y
6 print(x * y) # La multiplication de x par y
7 print(x / y) # La division reel de x par y
8 print(x // y) # La division entier de x par y
9 print(x ** y) # x exposant y
10
```

❷ Opération de comparaison

```
1 # Operation comparaison
2 x=1
3 y=3.6
4 print(x < y) # inferiorite
5 print(x > y) # superiorite
6 print(x <= y) # inferieur ou egale , meme chose pour superieur ou egale
7 print(x != y) # difference
8 print(x == y) # egalite
9
```

❸ Opération de Logique

```
1 # Operation de Logique
2 print(True & True) # AND(et)
3 print(False | True) # OR(ou)
4 print(False ^ True) # XOR(ou exclusif)
5
```

Pour bien comprendre ce code, il est important de l'exécuter vous-même, en suivant chaque étape attentivement pour une meilleure assimilation avant de poursuivre notre aventure. N'oubliez pas d'ajouter chaque information que vous jugez utile dans votre boîte à outils de développeur. Et là, vous allez me dire : "C'est bon, on a compris la notion d'opération arithmétique. Si nous faisons l'addition de deux nombres, nous obtenons un nombre. Mais si nous effectuons une opération de comparaison, que devrions-nous obtenir?" En fait, c'est simple à comprendre. Disons $2 < 3$ (2 est inférieur à 3) : vrai ou faux? Et c'est tout, je pense que vous avez déjà la réponse. Une opération de comparaison nous renvoie soit vrai (**True**) soit faux (**False**). En autre terme on obtient un Booléen. Et c'est la même avec les opérations de logique. C'est trop beau n'est ce pas!

Vous pouvez combiner ces trois opérations pour créer des structures plus complexe et intéressantes :

Exemple :

```
1 # Declaration des variables
2 x,y=1,3
3 print((x+y) >= 3 & x >0)
```

3.2 Structures de contrôle : Conditions et Boucles

Nous allons maintenant explorer deux éléments fondamentaux de la programmation : les conditions et les boucles. Ces structures de contrôle permettent d'exécuter des morceaux de code de manière sélective ou répétée. Commençons par les conditions.

❶ Conditions

Une condition permet de tester si une expression est vraie ou fausse, et d'exécuter un bloc de code en conséquence. En Python, on utilise les instructions 'if', 'elif', et 'else' pour cela.

```
1  # Exemples de conditions
2  x = 10
3  y = 5
4
5  if x > y:
6      print("x est plus grand que y")
7  elif x == y:
8      print("x est égal a y")
9  else:
10     print("x est plus petit que y")
11
```

Ici, la condition 'if' vérifie si 'x' est plus grand que 'y'. Si c'est vrai, le premier bloc est exécuté. Si c'est faux, on passe à la condition 'elif' pour vérifier si 'x' est égal à 'y'. Enfin, si aucune des deux conditions précédentes n'est vraie, le bloc 'else' est exécuté.

Vous pouvez imaginer des conditions encore plus complexes en combinant plusieurs expressions à l'aide des opérateurs logiques ('and', 'or', etc.).

❷ Boucles

Les boucles permettent d'exécuter un morceau de code plusieurs fois, soit pour un nombre déterminé de répétitions, soit tant qu'une condition est vraie. Il existe deux types principaux de boucles en Python : la boucle 'for' et la boucle 'while'.

Boucle for :

La boucle 'for' permet de parcourir une séquence (comme une liste, une chaîne de caractères, etc.) et d'exécuter un bloc de code pour chaque élément de cette séquence.

```
1  # Boucle for
2  fruits = ["pomme", "banane", "orange"]
3  for fruit in fruits:
4      print(fruit)
5
```

Dans cet exemple, la boucle 'for' parcourt chaque élément de la liste 'fruits' et l'affiche. À chaque itération, la variable 'fruit' prend la valeur de l'élément suivant dans la liste.

Boucle while :

La boucle 'while' permet d'exécuter un bloc de code tant qu'une condition est vraie.

```
1  # Boucle while
2  i = 0
3  while i < 5:
4      print(i)
5      i += 1  # équivalent e i = i + 1
6
```

Ici, tant que la variable 'i' est inférieure à 5, la boucle continue de s'exécuter, et à chaque itération, la valeur de 'i' est incrémentée. Attention aux boucles infinies si la condition ne devient jamais fausse !

③ Combiner conditions et boucles

Vous pouvez bien sûr combiner des conditions et des boucles pour créer des programmes plus complexes. Par exemple, en utilisant une boucle avec une condition ‘if’ pour ne traiter que certains éléments :

```
1  # Combinaison de condition et boucle
2  for i in range(10):
3      if i % 2 == 0:  # Si i est pair
4          print(f"{i} est un nombre pair")
5      else:
6          print(f"{i} est un nombre impair")
7
```

Dans cet exemple, la boucle parcourt les nombres de 0 à 9, et pour chaque nombre, la condition ‘if’ vérifie si le nombre est pair ($i \% 2 == 0$). Si c’est le cas, le nombre est affiché comme pair, sinon il est affiché comme impair.

Maintenant que nous avons vu les bases des conditions et des boucles, vous devriez être en mesure de contrôler efficacement le flux d’exécution de vos programmes. En combinant ces structures, vous pouvez écrire des scripts plus dynamiques et interactifs. Continuez à expérimenter ces concepts dans vos propres projets et ajoutez-les à votre boîte à outils Python !